

Fundamentals of API Development

Security for APIs: Standards

Day 3 - Session 2

What We'll Cover

1. Why security standards matter: the cost of skipping them
2. Authentication, authorization, CORS (Cross-Origin Resource Sharing), and caller patterns
3. Secure authentication defaults and least privilege
4. Rate limiting: the token bucket algorithm
5. Demo and Lab 2.2: applying basic API security rules
6. Lifecycle & security deep-dive, and Day 3 wrap-up

Why This Isn't Optional

APIs are most of the attack surface now

The Cost of Skipping Standards

APIs now carry the large majority of web traffic, and API-targeted attacks have surged roughly 680% since 2021. The average data breach involving an API is cited at \$4.88 million.

IMPORTANT

The OWASP (Open Worldwide Application Security Project) API Security Top 10 is the industry-standard reference here. Broken Object Level Authorization, Broken Authentication, and Lack of Resources & Rate Limiting map directly onto today's three topics.

OWASP (Open Worldwide Application Security Project): The #1 API Risk

API-targeted attacks have surged roughly 680% since 2021.

Which OWASP API Security Top 10 category is the most commonly exploited?

- A) Broken Object Level Authorization (BOLA), accessing objects you shouldn't
- B) Broken Authentication, impersonating another user
- C) Excessive Data Exposure, APIs returning more data than the client needs
- D) Lack of Resources & Rate Limiting, no cap on how often you can call

Chat Check

Type your answer (A, B, C, or D) in the chat window now!

Note: **BOLA** stands for Broken Object Level Authorization.

OWASP: The Answer

A, Broken Object Level Authorization (BOLA) is consistently #1.

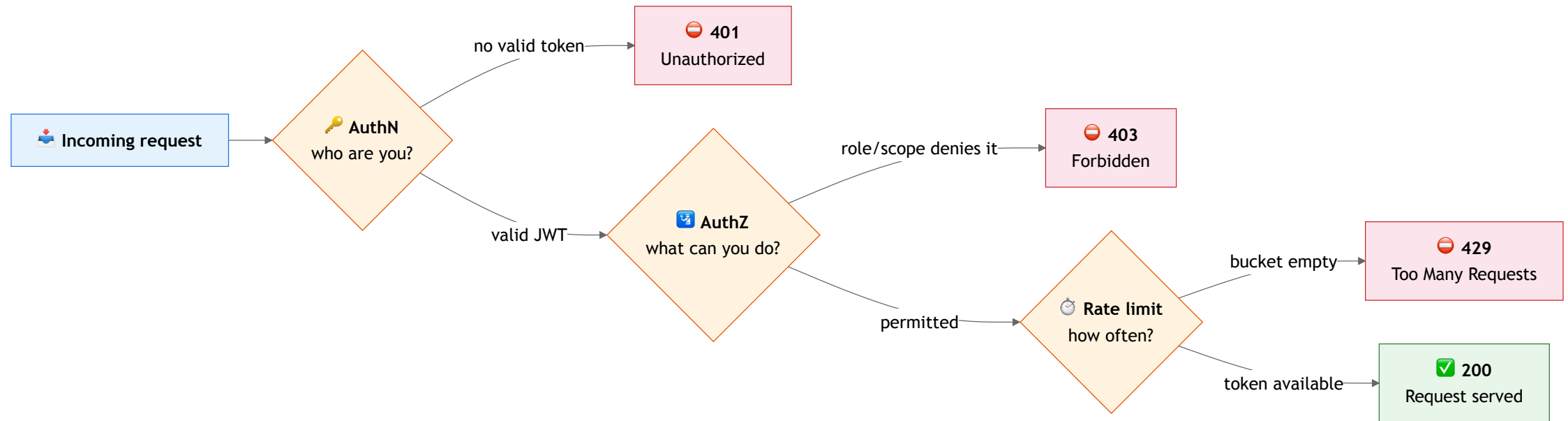
BOLA happens when an API checks *that* you're authenticated but not *whether* you should access object ID 42. Simply changing `/accounts/42` to `/accounts/43` in the URL should not grant access to someone else's data.

The OWASP API Security Top 10 maps directly onto today's three topics: authN, authZ, and rate limiting.

Three Questions, Three Standards

Who are you, what can you do, how often can you do it

The Request Gauntlet



Authentication vs Authorization: Discussion

A user logs into their banking app successfully and gets a valid token. They then try to view the balance of an account belonging to someone else.

Is this a failure of Authentication or Authorization?

DISCUSS

What specific question does each mechanism answer?

Authentication vs Authorization: The Answer

	Authentication	Authorization
Question	Who are you?	What can you do?
Mechanism	OAuth 2.0 / OIDC (OpenID Connect), short-lived JWT (JSON Web Token)	RBAC (Role-Based Access Control), ABAC (Attribute-Based Access Control), or scope-based access
If skipped	Anyone can call the API as anyone	An authenticated caller acts outside their scope

WARNING

Store refresh tokens in HTTP-only, Secure, SameSite cookies, never in localStorage, which is vulnerable to XSS (Cross-Site Scripting)-based token theft.

Authentication vs Authorization (Fintech)

REAL-WORLD SCENARIO

Fintech (Navy Federal — RBAC Implementation):

- A member authenticating to the Navy Federal app receives a short-lived JWT, answering "who are you."
- A separate authorization layer (RBAC) then reads the token's scopes to ensure a standard user cannot access administrative branch APIs, answering "what can you do."

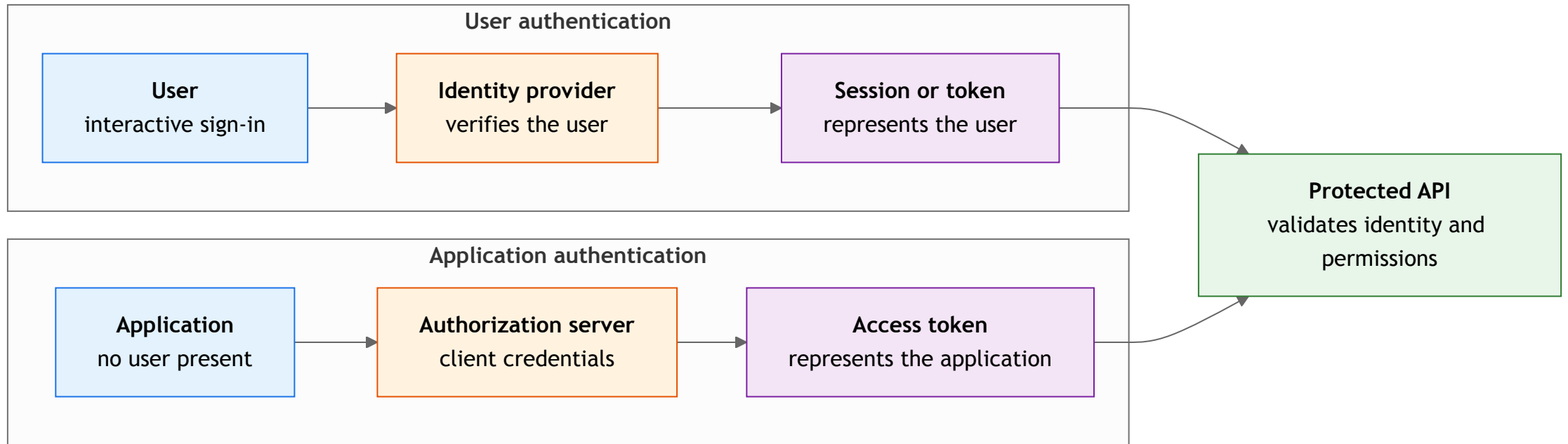
Authentication vs Authorization (Retail)

REAL-WORLD SCENARIO

Retail (Target — Vendor Access):

- Target requires third-party logistics vendors to authenticate using OAuth 2.0 client credentials.
- Even with a valid token, the API's authorization layer strictly enforces that a vendor can only update shipment statuses for their specific vendor ID, preventing BOLA attacks.

Users and Applications Authenticate Differently



A user credential represents a person. A client-credentials token represents the calling application, with no user present.

Secure Authentication Defaults

- Validate a token's signature, trusted issuer, intended audience, and expiration before trusting its claims
- Apply roles or scopes only after authentication succeeds
- Keep credentials outside source control and rotate them when exposed or expired
- Give each caller only the permissions it needs, for only as long as it needs them

Authentication establishes identity; least-privilege authorization limits what that identity can do.

CORS : Browser Access to APIs

- Browsers enforce the same-origin policy; CORS (Cross-Origin Resource Sharing) lets an API name trusted web origins
- A preflight **OPTIONS** request checks permitted methods and headers before some requests
- Allow only the origins, methods, and headers the client needs
- Do not combine credentialed requests with an unrestricted ***** origin

CORS is a browser access policy, not authentication or authorization.

Demo: The Authentication Gate

From `day3/demos`, run `java -cp target/classes com.kavinschool.demos.DemoJwtAuthNGate`.

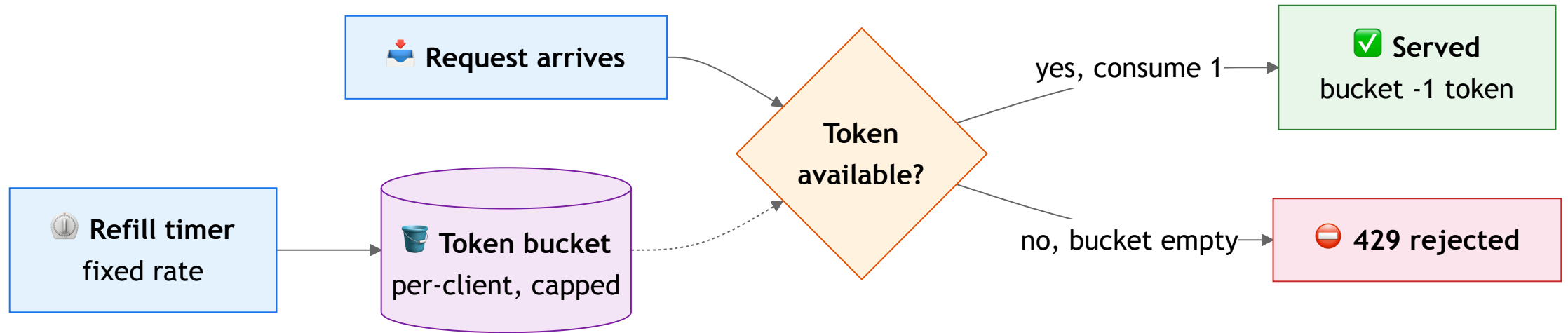
- Four requests: a valid token, an expired one, a tampered one, and none at all
- Each is gated on authentication before anything else runs
- Only the valid token reaches "authenticated," the rest fail with `401`

Offline, no API key, no network. Full OAuth 2.1 is out of scope for a 30-minute lab, so Lab 2.2 uses a simplified authenticated-principal setup and lab time goes to authorization and rate limiting.

Rate Limiting

Absorb a burst, never exceed the average

The Token Bucket



Bucket4j: The Java Standard

The token bucket algorithm allows short bursts while enforcing a steady long-term rate. Bucket4j is the common Java library for it.

The terms often overlap: rate limiting defines the allowed request rate, while throttling enforces it by delaying or rejecting requests. HTTP APIs commonly reject with `429` and a `Retry-After` hint.

CAUTION

An in-memory bucket per instance means N instances behind a load balancer silently multiply the real limit by N. A shared store (Redis) is what makes a limit real across a cluster.

The Rate Limiter Trap

You configure a rate limit of 100 requests/minute per user. Your API runs behind 4 instances with in-memory buckets.

What is the real limit a single user can hit?

- A) 100 requests/minute, the configured limit
- B) 400 requests/minute, 4 instances × 100 each
- C) 25 requests/minute, the load balancer divides the limit
- D) Unpredictable, it depends on which instance handles each request

Chat Check

Type your answer (A, B, C, or D) in the chat window now!

The Rate Limiter Trap: The Answer

B, 400 requests/minute. Each instance has its own bucket.

An in-memory bucket per instance means N instances silently multiply the real limit by N . A load balancer distributing requests across 4 instances gives each one its own independent counter.

The fix: a shared store like Redis makes the limit real across the whole cluster.

Rate Limiting (Fintech)

REAL-WORLD SCENARIO

Fintech (Navy Federal — Fraud Prevention):

- To prevent credential stuffing attacks on login endpoints, Navy Federal uses a distributed rate limiter backed by a shared Redis cluster.
- This ensures that an attacker cannot bypass the limit simply by having their traffic routed to different backend instances by the load balancer.

Rate Limiting (Ecommerce)

REAL-WORLD SCENARIO

Ecommerce (Wayfair — Flash Sales):

- During major holiday flash sales, Wayfair applies strict token bucket rate limiting on their checkout APIs to absorb massive traffic spikes.
- This prevents the core database from becoming overwhelmed, ensuring that the system gracefully rejects excess traffic with **429** rather than crashing entirely.

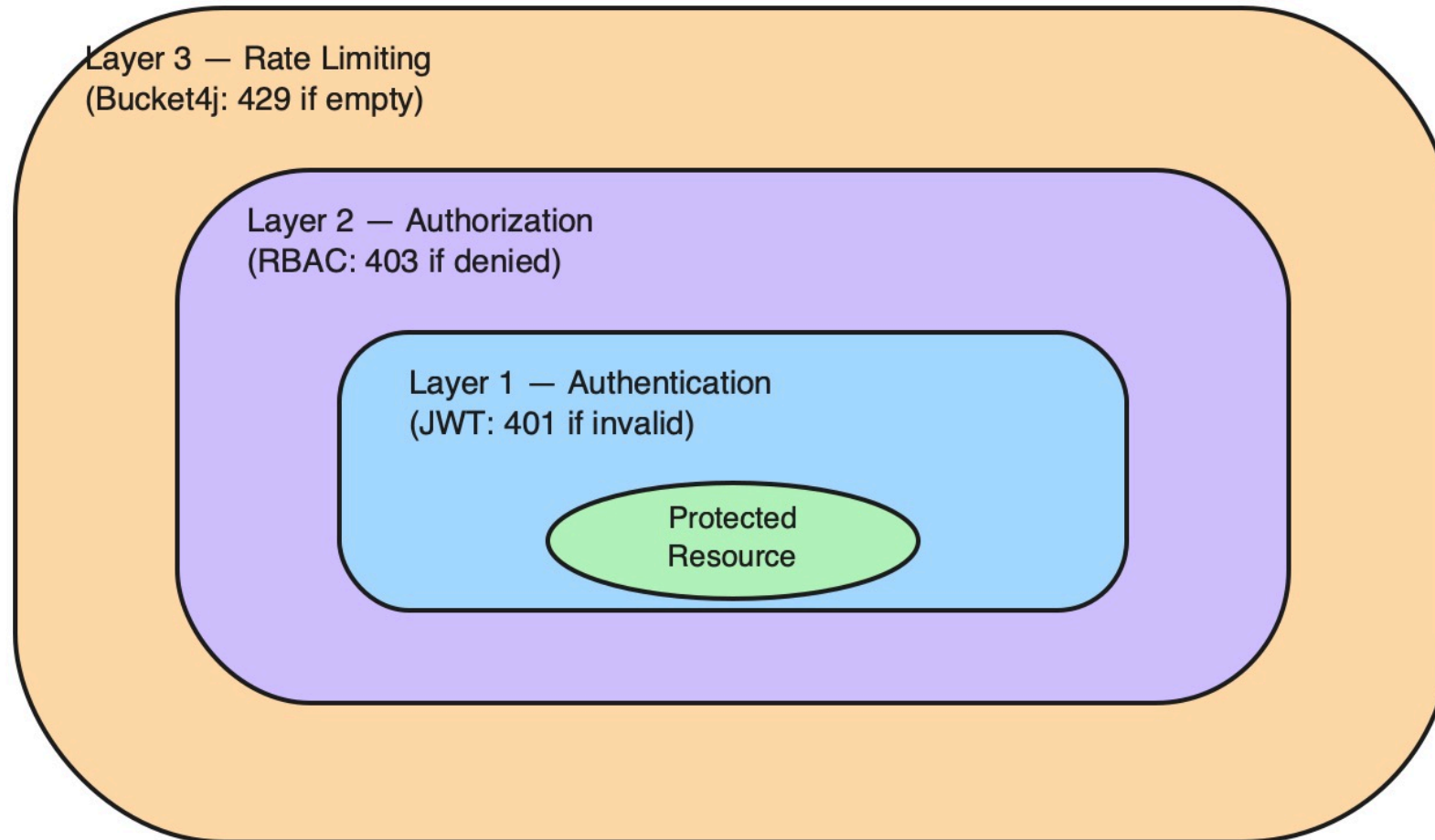
Demo: Token Bucket, From Scratch

From `day3/demos`, run `java -cp target/classes com.kavinschool.demos.DemoTokenBucketRateLimiter`.

- Capacity 3, refill rate 1 token/second, a simulated clock
- A burst of 5 requests: 3 served, 2 rejected with `429`
- After 2 simulated seconds, 2 more requests are served

Offline, no API key, no network.

Defense in Depth, All Three Layers



Lab 2.2: Implementing Basic API Security Rules

Open `day3/lab2_2_implementing_basic_api_security_rules/` in the companion repo.

Goal: Implement or configure basic security rules on the shared Spring Boot API: an authenticated principal, an authorization rule, and a rate limit.

1. Configure Spring Security so requests carry an authenticated principal
2. Add a `@PreAuthorize` rule restricting one endpoint to a role
3. Add a simple in-memory rate limit to the same endpoint

Stretch: add rate-limit tiers; validate auth flows with a Postman collection.

Lifecycle & Security, Together

Same discipline, applied twice today

Group Review: Lab 2.1 and Lab 2.2 Outputs

Discuss with your group, using evidence from your own labs:

1. Where did your Lab 2.1 version strategy already anticipate a security concern?
2. Which of today's standards would have caught a real incident you've heard of?
3. What's the cost of skipping lifecycle discipline versus skipping security discipline?

Running behind? Use this as catch-up time for Lab 2.1/2.2.

Key Takeaways

1. Authentication, authorization, and rate limiting answer separate questions, reflected in the OWASP API Security Top 10.
2. CORS controls browser origins; it does not authenticate or authorize a caller.
3. User sessions or tokens represent people, while client-credentials tokens represent applications.
4. Validate tokens, protect credentials, and grant every caller the least privilege it needs.
5. Token buckets absorb bursts, but clustered rate limits need shared state to enforce one real limit.

Day 3 Wrap-Up

Next: Day 4 - DevOps, API Ecosystem & Capstone

What You Have Now

From today	Feeds
A non-breaking version strategy for the shared API	Day 4's CI/CD needs a stable versioning story to automate around
Basic security rules (authN, authZ, rate limiting)	Day 4's test suite needs to cover the security rules you just added
The OWASP API Security Top 10 framing	Day 4's "API security within the ecosystem" builds on this directly

Day 4 Preview

Tomorrow the pieces from all three days come together: design, docs, versioning, security, and testing, in one capstone.

1. DevOps in API development: CI/CD, automated testing
2. The API ecosystem: API First strategy, security within it
3. Card applications API capstone: lifecycle, Spring Security, rate limiting, OpenAPI, and JUnit5
4. Capstone solution review and course synthesis

Bring your Lab 2.1/2.2 work. Day 4's single capstone builds directly on it.

Questions?

See you tomorrow